

Particle Swarm Optimization based on Dynamic Island Model

Houda Abadlia
COSMOS Laboratory, National School
of Computer Sciences, University of
Manouba, Tunisia
houdaabadlia@gmail.com

Nadia Smairi
COSMOS Laboratory, National School
of Computer Sciences, University of
Manouba, Tunisia
nadia.smairi@gmail.com

Khaled Ghedira
COSMOS Laboratory, National School
of Computer Sciences, University of
Manouba, Tunisia
khaled.ghedira@isg.rnu.tn

Abstract—Particle Swarm Optimization (PSO) algorithm is a metaheuristic that has been used for solving optimization problems. In this method, many modifications have been carried out in order to improve the search performance. Furthermore, island models is a structured population mechanism used to preserve the diversity and thus to improve the population performance. The aim of this paper is to integrate dynamic island models with PSO algorithm to improve its convergence and its diversity properties where the new method is referred to as island PSO. The dynamic regulation of migrations aims to distribute the particles in the search space. The experimental results, using a set of benchmark functions show that the island model context is crucial to the PSO performance and the comparative study shows the efficiency of the integration of dynamic island models.

keywords—Particle Swarm Optimization; Dynamic Island Model; Migration; Diversity

I. INTRODUCTION

PSO, a recent Evolutionary algorithm, was proposed by Kennedy and Eberhart for solving complex problems. It utilizes coordinated particles to explore the search space. Particles correspond to the search points. Each particle is initialized with a random position and a random initial velocity in the search space. The position and the velocity of each particle are updated along with the search process in an intelligent way, based on its own experience and on the experience of its neighbors. PSO has the ability to quickly converge, which can lead to a stagnation on a local optimum. In the literature, there is no way to ensure that an optimization method will give the best results for all the possible instances of a given problem. In practice, you can always default to one method over another. The hybridization can be regarded as an effective mean of finding a compromise between the advantages and the disadvantages of several optimization methods. In this paper, we are proposed to improve the Standard PSO algorithm by combining it with a dynamic island models.

This paper is organized as follows: Sect. 2, the background to the particle swarm optimization paradigm, and the island models. In Sect. 3, the proposed algorithm is described. In Sect. 4, an experimental study is presented using CEC'05 and SPSO'11 algorithms and benchmarks functions. Then, the conclusion is presented in Sect. 5.

II. BACKGROUND

The global optimization problems are normally formulated in terms of objective function as follows:

Optimize $f(x)$

subject to $\begin{cases} g_j(x) \leq 0, j = 1 \text{ to } m \\ h_l(x) = 0, l = 1 \text{ to } L \end{cases}$

where $f(x)$ is the objective function and $x_i = [x_1, x_2, \dots, x_{Dim}]$ where Dim is the dimension of the search space. The global optimization problems can be associated with a number of inequality and equality constraints (respectively g_j and h_l , m is the number of inequality constraints and L is the number of equality constraints).

A. Particle Swarm optimization

Particle swarm optimization is a metaheuristic proposed in 1995 by Kennedy and Eberhart [1]. PSO is a population-based stochastic approach for solving continuous and discrete optimization problems. The concept under PSO is to emulate the social interaction behavior of birds flocking and fish schooling. It utilizes a “population” of particles that fly through the problem hyperspace with given velocities. At each iteration, the velocities of the particles are stochastically adjusted according to the influence of its best solution and the influence of the best solution of its neighbors, then computes a new point to be evaluated. The displacement of a particle is influenced by three components:

- Physical component: the particle tends to keep its current direction of displacement;
- Cognitive component: the particle tends to move towards the best explored site until now;
- Social component: the particle tends to rely on the experience of its congeners, then moves towards the best site already explored by its neighbors.

The displacement of each particle in the search space is based on its current position and the update of its velocity. Let $x_i(t)$ be the position of the particle p_i at the time step t . The new position of the particle p_i is obtained by the addition of the speed $v_i(t)$ of the current position:

$$x_i(t) = x_i(t-1) + v_i(t) \quad (1)$$

Each particle in the swarm, changes its velocity according to two essential directions. The first direction is related to his personal experience, which is the best position found by the particle during the search process, which is noted p_{best} . The second direction is the best position found by the whole swarm. This information is obtained from the

knowledge of how the other particles performed their searches.

The principle change of the velocity is defined as follows:

$$v_i(t) = wv_i(t-1) + r_1c_1(x_{pbest_i} - x_i(t)) + r_2c_2(x_{gbest} - x_i(t)) \quad (2)$$

The $v_i(t)$ is the velocity of the i th particle ($i \in 1, 2, \dots, s$) where: s is the population size c_1, c_2 are the learning factors that will be fixed throughout the whole process, called acceleration coefficients; r_1, r_2 are two random numbers in the range $[0, 1]$ selected uniformly for each dimension at each iteration $wv_i(t-1)$ is the physical component, $r_1c_1(x_{pbest_i} - x_i(t))$ is the cognitive component, where c_1 controls the cognitive behavior of the particle, and $r_2c_2(x_{gbest} - x_i(t))$ is the social component, where c_2 controls the social behavior of the particle.

To improve the performance of this algorithm, there are many interesting researches on hybridizations of PSO. So the evolutionary operators, such as selection, crossover, and mutation, have been introduced to keep the best solution. This technique copies the current positions and velocities of the best half of the population, to replace the worst half without changing the best values of any of the particles in the current step. Nakano and al. proposed a PSO approach based on the concept of tabu search. The population is divided into two swarms in this algorithm. The first swarm deals with intensification and the second one deals with diversification. The authors in [2] proposed a new PSO method called AugPSO that uses two strategies: boundary-shifting and particle-position-resetting. The aim of the algorithm is to optimize lattice structures. The boundary-shifting technique forces the displacement of particles towards the boundary of feasible and unrealizable regions in order to increase the convergence rate in the search space. The technique of particle-position-resetting is motivated by the mutation scheme inspired by genetic algorithms, in order to increase the diversity.

Algorithm: The original Particle Swarm Optimization (PSO)

```

Initialize randomly the position  $x$  and the
velocity  $v$  of each particle : set iteration=1
For  $i=1$  to  $s$  do
    pbest $i$ = $x_i$ 
End for
While the stopping criterion is not satisfied
do
    for  $i=1$  to  $s$  do
        Update position  $x_i$  using (1) and (2)
        Calculate particle fitness  $f(x_i)$ 
        Update pbest $i$ , gbest
    end for
    set iteration=iteration+1
End while

```

Fig. 1. The Particle Swarm Optimization algorithm

Other interesting PSO variants proposed are multi-swarm algorithms. The authors proposed in [3] an algorithm based on a Master-Slave model, in which a population consists of one master swarm and several slave swarms. The slave swarms execute a single PSO independently to maintain the diversity of the particles, while the master swarm evolves based on its own knowledge and on the knowledge of the slave swarms.

B. Island models

Island models (IMs) is a multi-population approach used by evolutionary algorithms. It is inspired by the punctuated equilibrium model. The first studies of the IMs connected with the genetic algorithms showed a high efficiency and an equivalence between the exploitation and the exploration in the research space. In general, IMs divides the population into subpopulations. Each population is considered an island and is isolated from other islands during the evolutionary process. The islands evolve independently for a while and, then a process of migration is carried out by the islands which can exchange candidate solutions.

This migration process has the advantage of promoting or encouraging the development of subpopulations by introducing new characteristics through the exchange of candidate solutions between the islands. The fundamental aspect of IMs is the collaboration between the islands, it is carried out by a periodic migration process. This process uses a strategy for which candidate solutions are transferred from one subpopulation to another. Migration allows subpopulations to help others by introducing new information in the form of migratory points.

There are several parameters specifying an island model, like:

- Islands number: represents the number of subpopulations in the model;
- Migration topology: represents the communication structure of the model;
- Migratory frequency: determines how often migration occurs;
- Migratory rate: determines how many individuals migrate from a subpopulation to another;
- Synchronization type: represents the type of synchronization for the migration process (synchronous or asynchronous);
- Migratory policy: defines which individuals will be removed and replaced when migration occurs;
- Migratory flow: defines the path of the emigrants inside the communication structure.

Most of time the island models are using a static way by introducing their parameters such as the immigrant replacement policy, the topology of the communication among subpopulations, and the synchronous or asynchronous nature of the connection among

subpopulations. The authors proposed a dynamic model to improve its performance [4] [5].

In the literature, several EA-based methods have integrated the island model concepts in order to improve the population diversity during the search and establish a balance between exploration and exploitation, Island based Genetic Algorithm [6], Island based Differential Evolution [7, 8], Island models based Ant Colony [9]. Moreover, the sensitivity of island based methods to their parameters such as number of islands, migration interval and migration frequency have been studied to show their effect on the convergence and values to improve the performance [10].

III. OUR PROPOSED APPROACH

The DIMPSO algorithm starts the search by several solutions divided into sub-swarms $E_1, E_2, \dots, E_n$. The sub swarms are arranged in a T topology migration and the distributed nodes are connected by arcs. Each sub-swarm is composed of a set of particles, each particle has a position in the search space denoted by $x_i = x_{i1}, x_{i2}, \dots, x_{id}, \dots, x_{iDim}$ and it remembers its best position encountered during the search process. Each particle position defines a solution to the problem being treated. The steps of the approach are described as follows:

Step 1:

The algorithm begins by initializing the various parameters: the number of islands that represent the sub-swarms (n), the size of each island I_s (N) and $\sum_{i=1}^n I_s$ represented the DIMPSO algorithm.

In each island or "sub-swarm", random positions denoted by $(x_{il} = x_{il1}, x_{il2}, \dots, x_{ild}, \dots, x_{ilDim})$ are initialized with their velocities denoted by $v_{il} = (v_{il1}, v_{il2}, \dots, v_{ild}, \dots, v_{ilD})$ in the search space of dimension D . Then we initialize the best position encountered by each particle (denoted by $pbest_{ild} = pbest_{il1}, pbest_{il2}, \dots, pbest_{ild}, \dots, pbest_{ilDim}$) by its initial position x_{ild} , where l is the island index.

Step 2:

Then, based on the objective function, the algorithm calculates the quality (fitness) of each particle in order to retrieve the best position encountered by each island (denoted $gbest_1 = gbest_1, gbest_2, \dots, gbest_1, \dots, gbest_n$). At each iteration, a new position is calculated for each particle in the swarm according to its current position, its best position and the global best position encountered by the sub-swarm then the particles move according to the equations (1) and (2) : After creating new solutions, the algorithm proceeds to evaluate (i.e. calculate the fitness f for each new particle) and to update the values of $pbest_{id}$ and also $gbest_d$, as shown in formulas (1) and (2), respectively.

If ($f(pbest_{il})$) is better than $f(gbest_1)$ So $gbest_1 = pbest_{il}$. (2)

With: $i = 1, \dots, N$ (N represents the size of sub-swarm).

Step 3:

The dynamic island model can be defined by the following elements:

- Dimensions: a set of n islands $I = \{I_1, \dots, I_n\}$, a set of PSO algorithms, a set of sub-swarms $E = E_1, E_2, \dots, E_n$, each E_i is a subset of S . Each sub-swarm E_k is assigned to island I_k .
- A topology migration T can be represented by an undirected graph (I, V) where $V \subseteq I \times I$ is a set of edges between islands of I .
- A migration policy can be represented a squared matrix M of size m , such that $M(i, j) \in [0, 1]$ represents the probability for an individual to migrate from island i to island j . i.e., the initialization of M noted by $\forall (i, j) \in I^2 \setminus V, M(i, j) = 0$.

The used dynamic migration policy evolves during the search. Each island is equipped with two main processes that define how its sub-swarm evolves and how the migration policy is controlled. Those parameters are modified during the run according to incoming feedback received from other islands. This feedback allows the island to know how its particles have been improved on other islands. The learning component consists in sending more individuals to island that have been able to improve them significantly and less to islands that are currently less efficient for these particles. The selection component uses the migration matrix M to select the particles that are sent to other islands. The analyze component consists to evaluate the improvements of the particles after the PSO is applied. It sends this feedback information to the island where these individuals were originated from. The figure bellow represents the dynamic island mechanism for the migration policy and we define the following basic processes and data structures :

- A data matrix D is a square matrix of size n . $D(i, j)$ evaluates the improvement obtained by the particles of island i when they are processed on island j .
- An algorithm PSO is a process that computes a new swarm E_{t+1} from a given population E_t by applying an evolution process.
- $learn(D, D_{in})$ is a process that modifies D according to information received in D_{in} . In fact this process can be seen as an operation $D \leftarrow D$ and D_{in} .
- $update(M, D)$ is a process that modifies the migration matrix M according to the data matrix D . In fact the migration matrix is updated as: $M(i, k) = (1 - \beta)(\alpha M(i, k) + (1 - \alpha)R(k)) + \beta N(k)$ where n is a stochastic vector such

that $\|n\| = 1$. The parameter α represents the importance of the knowledge accumulated during the last migrations and β is the amount of noise, which is necessary to explore alternative search space areas by means of particles. The parameter R represents the intermediate reward vector which is used as an intensification strategy according to the update process using the data matrix D where only the line corresponding to island i is modified and the particles of the island i which have obtained the best improvement are rewarded:

$$R(k) = \begin{cases} \frac{1}{|B|} & \text{if } k \in B \\ 0 & \text{otherwise} \end{cases}$$

with $B = \operatorname{argmax}_k D(i, k)$

- $\operatorname{select}(P, M, j)$ is a process that selects a sets of particles from the current sub-swarm of the island E to be sent to another island j according to migration matrix M .
- $\operatorname{analyze}(E)$ is a process that evaluates the improvement obtained in island i by introducing particles from island k during the last iteration of the algorithm ($D_{out}(\kappa, i)$).

IV. EXPERIMENTAL RESULTS

In order to evaluate the proposed DIMPSO, 25 test functions with different characteristics have been used ($f_1 - f_{25}$), they are selected from the set of test functions of IEEE-CEC2005. A brief summary of the test functions used in our experiments is provided in Table III where each test function is abbreviated and its variables value range are recorded as appears in the original source [11]. The dimension D of each solution vector used in the experiments is also recorded together with the fitness value of the optimal solution $f(x^*)$.

The set of IEEE-CEC2005 test functions ($f_1 - f_{25}$) can be described as follows: the first 5 functions ($f_1 - f_5$) are unimodal and shifted, the second 7 test functions ($f_6 - f_{12}$) are basic multimodal and shifted; the third two functions ($f_{13} - f_{14}$) are expanded multimodal; and the fourth 11 functions ($f_{15} - f_{25}$) are hybrid composition. All of them are non separable, rotated and multimodal functions containing a large number of local minima. We follow the parameters and conditions of the CEC competition [11] where the 25 repeated runs have been performed for each test function. The 25 runs have been summarized in terms of the error average (EA) of the best individual, $EA = |f(x^*) - f(x^{best})|$, noted that x^* is a given optimal solution while the x^{best} is the average best solution obtained in 25 runs.

Obviously, based on some interpreted results for island GA, it is assumed that the number of islands and island size are more important and they are studied first. We consider

reasonable values on the basis [12]. In our case, we vary the following number of islands $\{n=2, 5, 10\}$ values have been studied and the values of α and β are respectively 0.8 and 0.01. Table I shows the results or 25 functions of CEC-2005 using 10 dimensions. The values presented in the table concern the results of the DIMPSO and other evolutionary algorithms produced after 25 runs. The best solutions produced are highlighted in bold (lowest is best).

As apparent in the table, the higher the value of I_n is almost the better. In other words, when the value of I_n is small, the particles in the DIMPSO will not sub-divided into several islands and thus few search space regions will be simultaneously explored. However, a large value of I_n leads to a better diffusion on the swarm. Using a smaller number would probably cause too much granularity of each island. It should be emphasized that although increasing the value of I_n improves the results produced, it also increases the computational complexity of the algorithm. To compare the proposed approach DIMPSO with other state of-the-art, eight comparative methods are used such as comparative approaches contain the SPSO'11 [13]. The PSO parameters are the same parameters used on SPSO'11 a swarm size equal to 40 particles for 10 Dim; random initialization of particle positions and velocities; random topology with $K=3$ informants; acceleration coefficients c_1 and c_2 equal to $0.5 + \ln(2)$; and a constant inertia weight equal to $\omega = 1/(2 * \ln(2))$.

The table I presents the statistical test for the proposed approach with three different number of islands, according to it, the convergence rate of DIMPSO is better than the SPSO algorithm, that's could be explained by the migration process using a dynamic regulation of migrations which distributes the particles on the most promising islands according to recent background information. At every stage of the search, the more efficient operators receive the greatest part of the computational resources, while the model is able to auto-adapt to the attractive power of each islands.

For example, the first three unimodal functions show a successful rate according to the found average error and for the second 3 functions are harder than the first ones despite that the fact DIMPSO gives the best results among all the studied algorithms. The function 8 is a shifted rotated Ackley's function with global optimum on bounds, which has a very narrow global basin and half of the dimensions of this basin are on the bounds. Hence, the search is almost like seeking a needle in a haystack. But the proposed approach shows that even with hard function, it can give as good results as the others algorithms. For the rest of the test functions, the proposed approach shows a better results compared to the others algorithms. The use of different number of islands could be the advantage to obtain a good results which the swarm divided into sub-swarms, so that improve the converge and the diversity for the PSO

algorithm. We are interesting to compare our approach with SPSO and other evolutionary algorithm from CEC'05 in table IV. Let us note that the best results were produced by DIMPSO for the majority of the test functions.

The figures 3 and 4 represent the position evolution of the best position of the swarm in terms of the number of

iterations for the test functions f_8 and f_9 , in the three cases according to the number of islands. They show that the best particle converge to a position in the search space.

TABLE I. AVERAGE ERROR RATE OBTAINED BY DIMPSO AND SPSO'11

Test function	Case 1 $n = 2$	Case 2 $n = 5$	Case 3 $n = 10$	SPSO'11
f_1	1.00E-009	0.0	0.0	1.00E-009
f_2	2.14E-008	0.0	0.0	1.00E-009
f_3	1.17E-009	0.0	0.0	1.00E-009
f_4	9.94E-009	0.0	0.0	1.52E-004
f_5	1.92E-005	0.0	0.0	1.6E-005
f_6	1.00E-009	1.41E-10	1.35E-11	4.2E-007
f_7	4.21E-002	4.42E-003	3.36E-001	5.1E-002
f_8	2.01E+001	2.02E+001	1.86E+001	1.94E+001
f_9	1.9E-001	1.84E-001	2.25E-001	1.17E-009
f_{10}	1.74E+000	1.68E+000	1.7E+000	3.2E+000
f_{11}	2.77E-001	2.9E-002	1.35E-001	2.31E+001
f_{12}	9.9E-001	9.87E-001	8.9E-001	1.89+000
f_{13}	1.84E+000	1.09E+000	1.14E+000	1.89E-001
f_{14}	2.95E+000	3.07E+000	4.3E+000	3.65E+000
f_{15}	9.11E+000	5.32E+000	4.79E+000	4.29E+000
f_{16}	3.08E+000	3.32E+000	2.33E+000	7.12E+000
f_{17}	5.17E+000	3.04E+000	4.13E+000	2.21E+001
f_{18}	1.47E-006	6.00E-008	4.01E-007	2.67E+000
f_{19}	1.33E-006	1.8E-002	1.78E-005	4.21E+002
f_{20}	7.05E-007	4.00E-008	7.04E-007	4.74E+002
f_{21}	1.5E-006	5.8E-007	2.86E-007	2.37E+002
f_{22}	1.12E+001	8.62E+001	4.86E+002	5.45E+002
f_{23}	2.01E+000	5.33E+000	1.8E+000	4.21E+001
f_{24}	9.5E-001	9.54E-001	9.38E-001	1.09E+002
f_{25}	1.97E+001	1.9E+001	2.85E+001	2.56E+002

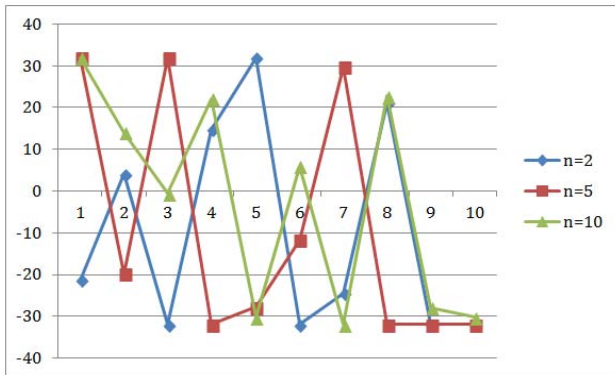


Fig. 2. Vector position of the best particle of the swarm for the Shifted Rotated Ackley's Function with Global Optimum on Bounds function (f_8) in dimension 10.

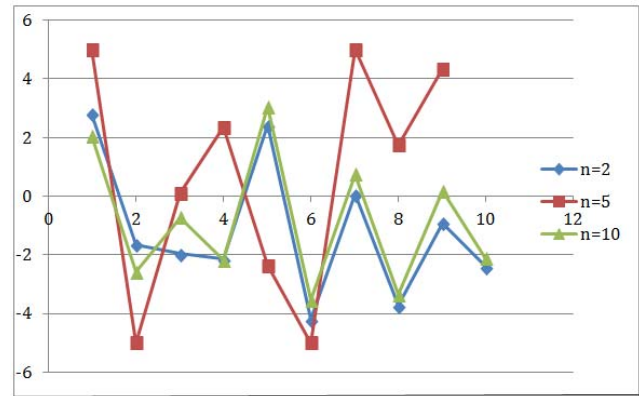


Fig. 3. Vector position of the best particle of the swarm for the Rastrigin function (f_9) in dimension 10.

TABLE II. THE TEST FUNCTIONS

Function	Name	Search range	D	$f(x^*)$
f_1	Shifted Sphere Function	$[-100, 100]$	10	-450
f_2	Shifted Schwefel's Problem 1.2	$[-100, 100]$	10	-450
f_3	Shifted Rotated High Conditioned Elliptic Function	$[-100, 100]$	10	-450
f_4	Shifted Schwefel's Problem 1.2 with Noise in Fitness	$[-100, 100]$	10	-450
f_5	Schwefel's Problem 2.6 with Global Optimum on Bounds	$[-100, 100]$	10	-310
f_6	Shifted Rosenbrock's Function	$[-100, 100]$	10	390
f_7	Shifted Rotated Griewank's Function without Bounds	$[0, 600]$	10	-180
f_8	Shifted Rotated Ackley's Function with Global Optimum on Bounds	$[-32, 32]$	10	-140
f_9	Shifted Rastrigin's Function	$[-5, 5]$	10	-330
f_{10}	Shifted Rotated Rastrigin's Function	$[-5, 5]$	10	-330
f_{11}	Shifted Rotated Weierstrass Function	$[-0.5, 0.5]$	10	90
f_{12}	Schwefel's Problem 2.13	$[-\pi, \pi]$	10	-460
f_{13}	Shifted Expanded Griewank's plus Rosenbrock's Function (F8F2)	$[-100, 100]$	10	-130
f_{14}	Shifted Rotated Expanded Scaffer's F6 Function	$[-5, 5]$	10	-300
f_{15}	Hybrid Composition Function	$[-5, 5]$	10	120
f_{16}	Rotated Version of Hybrid Composition Function F15	$[-5, 5]$	10	120
f_{17}	F16 with Noise in Fitness	$[-5, 5]$	10	120
f_{18}	Rotated Hybrid Composition Function	$[-5, 5]$	10	10
f_{19}	Rotated Hybrid Composition Function with narrow basin global optimum	$[-5, 5]$	10	10
f_{20}	Rotated Hybrid Composition Function with Global Optimum on the Bounds	$[-5, 5]$	10	10
f_{21}	Rotated Hybrid Composition Function	$[-5, 5]$	10	360
f_{22}	Rotated Hybrid Composition Function with High Condition Number	$[-5, 5]$	10	360
f_{23}	Non-Continuous Rotated Hybrid Composition Function	$[-5, 5]$	10	360
f_{24}	Rotated Hybrid Composition Function	$[-5, 5]$	10	260
f_{25}	Rotated Hybrid Composition Function without bounds	$[2, 5]$	10	260

V. CONCLUSION

This paper proposes a new hybrid approach between the PSO algorithm and the dynamic island model. The concept of the DIM consists into dividing the swarm into islands and the interaction between them using a dynamic regulation of the migration in order to improve the search space exploration. Moreover the use of higher number of island leads to better results due to better swarm diffusion. The results produced using the test functions CEC'05 show a good improvement in terms of convergence and diversity. As

the hybridization between dynamic island model with PSO algorithm was able to achieve good performance for global optimization functions, future work can be directed to:

- Investigating the effect of using other migration topologies with the dynamic migration policy.
- Applying our proposed approach for disturbed constraint optimization problem.

TABLE III. AVERAGE ERROR RATE OBTAINED IN CEC'2005 SPECIAL SESSION IN DIMENSION 10

	DE [14]	DMS-LPSO [15]	EDA [16]	G-CMA-ES [17]	KPCX [18]	LCMAES [19]	LSaDE [20]	SPC-PNX [21]
f ₁	1.00E-009	1.00E-009	1.00E-009	1.00E-009	1.00E-009	1.00E-009	1.00E-009	1.00E-009
f ₂	1.00E-009	1.00E-009	1.00E-009	1.00E-009	1.00E-009	1.00E-009	1.00E-009	1.00E-009
f ₃	1.94E-006	1.00E-009	2.12E+00	1.00E-009	4.15E-001	1.00E-009	1.67E-005	1.08E+00
f ₄	1.00E-009	1.89E-003	1.00E-009	1.00E-009	7.94E-007	1.76E+006	1.42E-005	1.00E-009
f ₅	1.00E-009	1.14E-006	1.00E-009	1.00E-009	4.85E+00	1.00E-009	1.23E-002	1.00E-009
f ₆	1.59E-001	6.89E-008	4.18E-001	1.00E-009	4.78E-001	1.00E-009	1.20E-008	1.89E+00
f ₇	1.46E-001	4.52E-002	4.20E-001	1.00E-009	2.31E-001	1.00E-009	8.26E-002	1.82E-001
f ₈	20.04E+001	20.00E+0	20.03E+0	20.00E+0	20.00E+0	20.00E+00	20.10E+0	20.02E+0
f ₉	9.55E-001	1.00E-009	5.42E+00	2.39E-001	1.19E-001	4.49E+001	1.00E-009	2.69E+00
f ₁₀	3.62E+000	5.29E+00	3.62E+00	7.96E-002	2.39E-001	4.08E+001	4.97E+00	7.30E+00
f ₁₁	8.47E-001	4.62E+00	3.94E+00	9.34E-001	6.65E+00	3.65+000	4.89+000	1.91E+00
f ₁₂	3.17E+001	2.40E+00	4.42E+00	2.93E+00	1.49E+00	2.09E+002	4.50E-007	2.60E+00
f ₁₃	9.77E-001	3.69E-001	1.84E+00	6.96E-001	6.53E-001	4.94E-001	2.20E-001	8.38E-001
f ₁₄	3.45E+000	2.36E+00	2.63E+00	3.01E+00	2.35E+00	4.01E+000	2.92E+00	3.05E+00
f ₁₅	2.59E+002	4.85E+00	3.65E+00	2.28E+00	5.10E+00	2.11E+002	3.20E+00	2.54E+00
f ₁₆	1.13E+002	9.48E+00	1.44E+00	9.13E+00	9.59E+00	1.05E+002	1.01E+00	1.10E+00
f ₁₇	1.15E+002	1.10E+00	1.57E+00	1.23E+00	9.73E+00	5.49E+002	1.14E+00	1.19E+00
f ₁₈	4.00E+002	7.61E+00	4.83E+00	3.32E+00	7.52E+00	4.97E+002	7.19E+00	4.40E+00
f ₁₉	4.20E+002	7.14E+00	5.64E+00	3.26E+00	7.51E+00	5.16E+002	7.05E+00	3.80E+00
f ₂₀	4.60E+002	8.22E+00	6.52E+00	3.00E+00	8.13E+00	4.42E+002	7.13E+00	4.40E+00
f ₂₁	4.92E+002	5.36E+00	4.84E+00	5.00E+00	1.05E+00	4.04E+002	4.64E+00	6.80E+00
f ₂₂	7.18E+002	6.92E+00	7.71E+00	7.29E+00	6.59E+00	7.40E+002	7.35E+00	7.49E+00
f ₂₃	5.72E+002	7.30E+00	6.41E+00	5.59E+00	1.06E+00	7.91E+002	6.64E+00	5.76E+00
f ₂₄	2.00E+002	2.24E+00	2.00E+00	2.00E+00	4.06E+00	8.65E+002	2.00E+00	2.00E+00
f ₂₅	9.23E+002	3.66E+00	3.73E+00	3.74E+00	4.06E+00	4.42E+002	3.76E+00	4.06E+00

REFERENCES

- [1] J. Kennedy and R. C. Eberhart. Particle Swarm Optimization. Proceedings of the IEEE International Conference on Neural Networks, IEEE Press. pp. 1942-1948, Perth, Australia, 1995.
- [2] Y. C. Lu, J. C. Jan, S. L. Hung and G. H. Hung. Enhancing particle swarm optimization algorithm using two new strategies for optimizing design of truss structures. Engineering Optimization Journal, Volume 4.5, 2013.
- [3] K. E. Parsopoulos. Parallel cooperative micro-particle swarm optimization: A master slave model. Journal of Applied Soft Computing, volume12, pp 3552-3579, 2012.
- [4] F. Lardeux and A. Goeffon. A Dynamic Island-Based Genetic Algorithms Framework. SEAL '10: 156-165, 2010.
- [5] C. Candan, A. Goeffon, F. Lardeux and F. Saubien. A Dynamic Island Model for Adaptive Operator Selection. GECCO'12, 2012.
- [6] F. Lardeux and A. Goeffon. A dynamic island-based genetic algorithms framework. In Proceedings of the 8th international conference on simulated evolution and learning. SEAL'10 pp 156-165. Berlin, 2010.
- [7] J. Kushida, A. Hara, T. Takahama, and A. Kido. Island-based differential evolution with varying subpopulation size. In 2013 IEEE sixth international workshop on computational intelligence & applications (IWCIA) (pp. 119-124). IEEE, 2013.
- [8] H. T. T. Thein. Island model based differential evolution algorithm for neural network training. Advances in Computer Science: An International Journal, 3(1), 2014.
- [9] R. Michel and M. Middendorf. An island model based ant system with look ahead for the shortest super sequence problem. In Parallel problem solving from nature PPSN V (pp. 692-701). Springer, 1998.
- [10] M. Tomassini. Spatially structured evolutionary algorithms: Artificial evolution in space time. Secaus, NJ, USA: Spring-Verlag New York, 2005.
- [11] P. N. Suganthan and al. Problem Definitions and Evaluation Criteria for the CEC'2005 Special Session on Real-Parameter Optimization,

Technical Report, Nanyang Technological University, Singapore, May 2005.

- [12] M. Tomassini. Spatially structured evolutionary algorithms: Artificial evolution in space time. Secaus, NJ, USA: Springer-Verlag New York, INC, 2005.
- [13] M. Zambrano-Bigiarini, M. Clerc and R. Rojas. Standard Particle Swarm Optimisation 2011 at CEC-2013: A baseline for future PSO improvements. IEEE Congress on Evolutionary Computation México, 2013.
- [14] J. Ronkkonen, S. Kukkonen and K.V. Price.. Real-parameter optimization with differential evolution. The 2005 IEEE congress on evolutionary computation CEC'2005, vol1, pp. 506-513, 2005.
- [15] J. J. Liang and P. N. Suganthan. Dynamic multi-swarm particle swarm optimizer with local search. The 2005 IEEE congress on evolutionary computation CEC'2005, vol1, pp. 522-528, 2005.
- [16] B. Yuan and M. Gallagher. Experimental results for the special session on real-parameter optimization at CEC 2005: A simple, continuous EDA. The 2005 IEEE congress on evolutionary computation CEC'2005 Vol. 2, pp. 1792-1799. IEEE, 2005.
- [17] Auger and N. Hansen. Performance evaluation of an advanced local search evolutionary algorithm. The 2005 IEEE congress on evolutionary computation (CEC'2005) (Vol. 2, pp. 1777-1784). IEEE, 2005.
- [18] A. Sinha, S. Tiwari and K. Deb. 2005. A population-based, steady-state procedure for real-parameter optimization. The 2005 IEEE congress on evolutionary computation (CEC'2005) (Vol. 1, pp. 514-521). IEEE, 2005.
- [19] A. Auger and N. Hansen. A restart CMA evolution strategy with increasing population size. The 2005 IEEE congress on evolutionary computation (CEC'2005) (Vol. 2, pp. 1769-1776). IEEE, 2005.
- [20] A. K. Qin and P. N. Suganthan. Self-adaptive differential evolution algorithm for numerical optimization. The 2005 IEEE congress on evolutionary computation (CEC'2005) (Vol. 2, pp. 1785-1791). IEEE, 2005.
- [21] P. J. Ballester, J. Stephenson, J. N. Carter and K. Gallagher. Real-parameter optimization performance study on the CEC-2005 benchmark with SPC-pnx. The 2005 IEEE congress on evolutionary computation (CEC'2005) (Vol. 1, pp. 498-505). IEEE, 2005.